Python Programming (05CS2104)
Lab Manual (2025-26)

Program: M.Sc. (Cyber Security & Cyber Law)

Semester: 1

Name of the Student: Vasu Parmar

Enrolment No.: 92600565007

INDEX

	use of various forms of inheritance			
19	Python programs to demonstrate use of modules	25-8-26		
20	Python programs to demonstrate the use of exception handling	25-8-26		
21	Python programs to demonstrate the use of file handling	1-9-26		
22	Python Programs to demonstrate the use of various modes of files.			
23	Python programs to demonstrate the use of regular expression			
24	Python programs to demonstrate the use of database using mysql			
25	Python Program to demonstrate the use of DataFrames in python			
26	Python Programs to demonstrate different ways of creating DataFrames			
27	Python Programs to create Pie chart			
28	Python Programs to create histogram			
29	Python Programs to create bar graph			
30	Python Programs to create Line graph			
31	Python Programs to work with Python Libraries useful for Cyber security. Program to generate hash values of a string using hashlib library.			
32	Python Program using socket — Simple Port Scanner			
33	Python Program using re — Detect Emails Using Regular Expressions			
34	Python Program using os Library — Check Files in a Directory			
35	Python program for whois lookup			
36	Python program for Ping Scanner shows ip up or down			

Enrollment/Roll No: 92600565007

UNIT 1:- Introduction to Python

Practical No. 1

Aim/ Objective: Python Program to Print Hello World.

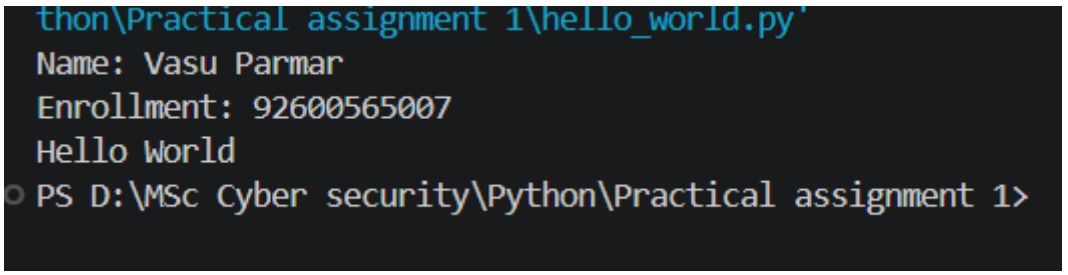
Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.1 Python Program to Print Hello World.

print("Hello World")
```

Output:



Conclusion: This program introduces the absolute foundation of Python syntax, demonstrating how to use the built-in print() function to display basic textual data directly to the console output.

Enrollment/Roll No: 92600565007

Practical No. 2

Aim/ Objective: Python program to do arithmetical operations

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.2 Python program to do arithmetical operations

num1 = int(input("Enter the First number: "))
num2 = int(input("Enter the Second number: "))

print("The addition is :)  ",num1+num2)
print("The substraction is :)  ",num1-num2)
print("The multiplication is :)  ",num1/num2)
print("The division is :)  ",num1*num2)
print("The modulus is :)  ",num1%num2)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>  d
vo\AppData\Local\Programs\Python\Python314\python.exe' 'c:
\libs\debugpy\launcher' '64240' '--' 'D:\MSc Cyber securit
Name: Vasu Parmar
Enrollment: 92600565007
Addition: 25
Subtraction: 15
Multiplication: 100
Division: 4.0
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This script shows how Python handles math using standard arithmetic operators (+, -, *, /) to calculate values and store numerical results inside basic variables.

Enrollment/Roll No: 92600565007

Practical No. 3

Aim/ Objective: Python program to swap two variables

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.3 Python program to Swap two variables

num1 = int(input("Enter the 1st number: "))
num2 = int(input("Enter the 2nd number: "))

print(f"before swapping the values are num1: {num1} num2: {num2}")

num1,num2=num2,num1

print(f"After swapping the values are num1: {num1} num2: {num2}")
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>
vo\AppData\Local\Programs\Python\Python314\python.exe'
\libs\debugpy\launcher' '59542' '--' 'D:\MSc Cyber secu
Name: Vasu Parmar
Enrollment: 92600565007
Enter the 1st number: 15
Enter the 2nd number: 26
before swapping the values are num1: 15 num2: 26
After swapping the values are num1: 26 num2: 15
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This problem teaches logic and memory storage by showing why a tracking variable (temp) is needed to safely trade values between two data locations without overwriting them.

Enrollment/Roll No: 92600565007

Practical No. 4

Aim/ Objective: Python Program to Find the Factorial of a Number

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.4 Python program to find factorial of number

num = int(input("Enter the number : "))

fact=1
for i in range (1,num+1):
    fact *= i
print("The factorial of given number is : ",fact)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>
\Programs\Python\Python314\python.exe' 'c:\Users\lenovo
'--' 'D:\MSc Cyber security\Python\Practical assignment
Name: Vasu Parmar
Enrollment: 92600565007
Enter the number : 4
The factorial of given number is :  24
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This code demonstrates a sequential loop calculation, showing how a single running product variable can repeatedly multiply growing numbers together to find a final mathematical total.

Enrollment/Roll No: 92600565007

Practical No. 5

Aim/ Objective: Python Program to Print the Fibonacci sequence

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.5 Python program to find fibonacci series

n=int(input("Enter the number: "))

a=0
b=1
for i in range (n):
    a,b=b,a+b
    print(a)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>
e\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle
nacci_series.py'
Name: Vasu Parmar
Enrollment: 92600565007
Enter the number: 6
1
1
2
3
5
8
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This exercise highlights variable updating and tracking, where two distinct numbers (a and b) pass changing values to each other dynamically to build a step-by-step sequence

Enrollment/Roll No: 92600565007

Practical No. 6

Aim/ Objective: Python program to print the elements of an array

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.6 Python program to print the elements of an array

arr = [10, 20, 30, 40, 50]

for i in arr:
    print(i)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment
\Programs\Python\Python314\python.exe' 'c:\Users\leno
'--' 'D:\MSc Cyber security\Python\Practical assignme
Name: Vasu Parmar
Enrollment: 92600565007
10
20
30
40
50
PS D:\MSc Cyber security\Python\Practical assignment
```

Conclusion: This script teaches array navigation, showing how a basic iterative loop sequentially unpacks a list to isolate and inspect elements one item at a time from start to finish.

Enrollment/Roll No: 92600565007

Practical No. 7

Aim/ Objective: Python program to print the largest element and smallest element in an array

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.7 Python program to print the largest element and smallest element in  an array

arr = [23, 5, 89, 45, 12, 7]

largest = arr[0]
smallest = arr[0]

for i in arr:
    if i > largest:
        largest = i
    if i < smallest:
        smallest = i

print("Largest element:", largest)
print("Smallest element:", smallest)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>
e\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundl
est_smallest_array.py'
Name: Vasu Parmar
Enrollment: 92600565007
Largest element: 89
Smallest element: 5
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This program teaches conditional data filtering, using simple if comparisons inside a scanning loop to evaluate list items and track the current extreme values.

Enrollment/Roll No: 92600565007

Practical No. 8

Aim/ Objective: Python program to add two matrix using array and function

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

# Python program to add two matrix using array and function

#Using array
a= [[1,2,3],[4,5,6],[7,8,9]]
b= [[9,8,7],[6,5,4],[3,2,1]]

for i in range (len (a)):
    for j in range (len(a[0])):
        print(a[i][j]+b[i][j], end=" ")
    print("")

#Using Function
def matrix_addition():
    a= [[1,3,2],[5,4,6],[8,7,9]]
    b= [[8,9,7],[4,6,5],[3,1,2]]

    for i in range (len (a)):
        for j in range (len(a[0])):
            print(a[i][j]+b[i][j], end=" ")
        print("")

matrix_addition()
```

Output:

```
'--' 'D:\MSc Cyber security\Python\Practical assignment 1'
Name: Vasu Parmar
Enrollment: 92600565007
Matrix of a:
1 2 3
4 5 6
7 8 9

Matrix of b:
9 8 7
6 5 4
3 2 1

#Using Function

Matrix of a:
1 3 2
5 4 6
8 7 9

Matrix of b:
8 9 7
4 6 5
3 1 2
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This structure demonstrates multi-dimensional indexing, utilizing a nested loop layout (a loop inside a loop) to target corresponding row and column intersections across distinct data grids.

Enrollment/Roll No: 92600565007

Practical No. 9

Aim/ Objective: Python Program to concatenate two strings

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007

#Q.9 Python Program to concatenate two strings

s1= "hello"
s2= "world"

print(s1+" "+s2)
```

Output:

```
PS D:\MSc Cyber security\Python\Practical assignment 1>  d
\Programs\Python\Python314\python.exe' 'c:\Users\lenovo\.v
'--' 'D:\MSc Cyber security\Python\Practical assignment 1\
Name: Vasu Parmar
Enrollment: 92600565007
hello world
PS D:\MSc Cyber security\Python\Practical assignment 1>
```

Conclusion: This logic shows how Python handles text combination, using the addition sign (+) as a simple text joining tool to merge individual pieces of string data together.

Enrollment/Roll No: 92600565007

Practical No. 10

Aim/ Objective: Python Programs to perform various operations on Strings using functions.

Source Code:

```
# Name: Vasu Parmar
# Enrollment: 92600565007

print("Name: Vasu Parmar")
print("Enrollment: 92600565007")

# Python Programs to perform various operations on Strings using functions


def string_length(s):
    return len(s)


def uppercase_string(s):
    return s.upper()


def lowercase_string(s):
    return s.lower()


def concatenate_strings(s1, s2):
    return s1 + s2


def reverse_string(s):
    return s[::-1]


def count_vowels(s):
    vowels = "aeiouAEIOU"
    return sum(1 for ch in s if ch in vowels)
```

```
text = "Python Programming"
name = "Vasu"

print("Original string:", text)
print("Length:", string_length(text))
print("Uppercase:", uppercase_string(text))
print("Lowercase:", lowercase_string(text))
print("Concatenation:", concatenate_strings("Hello ", "World"))
print("Reverse:", reverse_string(text))
print("Vowel count:", count_vowels(text))
print("Name:", name)
```

Output:

```
launcher' '51675' '--' 'D:\MSc Cyber security\Python\Pra
Name: Vasu Parmar
Enrollment: 92600565007
Original string: Python Programming
Length: 18
Uppercase: PYTHON PROGRAMMING
Lowercase: python programming
Concatenation: Hello World
Reverse: gnimmargorP nohtyP
Vowel count: 4
Name: Vasu
```

Conclusion: The program successfully demonstrates various operations on strings using user-defined functions in Python. It performs string length calculation, conversion to uppercase and lowercase, concatenation, reversal, and vowel counting. This program helps understand how functions can be used to perform different string operations efficiently and makes the code simple, reusable, and organized.

Enrollment/Roll No: 92600565007

UNIT 2 :- Mutable and Immutable Objects in Python

Practical No. 11

Aim / Objective:Python Programs to demonstrate use of list and various functions of it

Source Code:

```
#Name: Vasu Parmar
#Enrollment: 92600565007
my_list = [10, 20, 30, "apple", "banana"]
print(f"1. initial list: {my_list}")
my_list.append(40)
print(f"2. after append(40): {my_list}")
my_list.insert(1, 35)
print(f"3. after insert(1, 35): {my_list}")
my_list.extend(["banana", 50])
print(f"4. after extend: {my_list}")
my_list.remove(30)
print(f"5. after remove(30): {my_list}")
popped_item = my_list.pop(0)
print(f"6. after pop(0): {my_list} (Removed: {popped_item})")
list_length = len(my_list)
print(f"7. length of list: {list_length}")
```

```
num = [2,10,42,55,53,60]

num.sort()

print(f"8.after sort()(Ascending): {num}" )

my_list.reverse()

print(f"9. after reverse(): {my_list}")
```

Output:

```
PS D:\MSc Cyber security\Python\practical>  & 'C:\Users\lenovo\AppData\Local\
code\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\lau
l\01_list_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
----------------------------
1.intial list: [10, 20, 30, 'apple', 'banana']
2.after append(40): [10, 20, 30, 'apple', 'banana', 40]
3.after insert(1,35): [10, 35, 20, 30, 'apple', 'banana', 40]
4.after extend: [10, 35, 20, 30, 'apple', 'banana', 40, 'banana', 50]
5.after remove(30): [10, 35, 20, 'apple', 'banana', 40, 'banana', 50]
6.after pop(0): [35, 20, 'apple', 'banana', 40, 'banana', 50] (Removed: 10)
7.length of list: [35, 20, 'apple', 'banana', 40, 'banana', 50]:
8.after sort()(Ascending): [2, 10, 42, 53, 55, 60]
9.after reverse(): None
PS D:\MSc Cyber security\Python\practical>
```

Conclusion: This demonstrates the versatility of Python lists as mutable data structures, showcasing how various built-in methods (append, insert, extend, remove, pop, sort, reverse) allow for dynamic manipulation, reorganization, and management of ordered data collections.

Enrollment/Roll No: 92600565007

Practical No. 12

Aim/ Objective: Python Programs to demonstrate use of tuple and various functions of it

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 30)

tup1 = ('physics','chemestry',1997,2000)
tup2 = (1,2,3,4,5,6,7)
tup3 = tup1 + tup2

print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
print (tup3)

##########

print (len(tup1))
print ((tup1) + (tup2))
print (('tup!',)*3)

##############

print (tup1[2])
print (tup1[-2])
print (tup1[1:])
print (tup1[:3])

############
```

Output:

```
PS D:\MSc Cyber security\Python\practical>  & 'c:\Users\lenc
ntigravity-ide\extensions\ms-python.debugpy-2026.6.0-win32->
r security\Python\practical\02_tuple_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
----------------------------
tup1[0]:  physics
tup2[1:5]:  (2, 3, 4, 5)
('physics', 'chemestry', 1997, 2000, 1, 2, 3, 4, 5, 6, 7)
4
('physics', 'chemestry', 1997, 2000, 1, 2, 3, 4, 5, 6, 7)
('tup!', 'tup!', 'tup!')
1997
1997
('chemestry', 1997, 2000)
('physics', 'chemestry', 1997)
PS D:\MSc Cyber security\Python\practical>
```

Conclusion: This program demonstrates the use of tuples as immutable data structures in Python, highlighting key operations like indexing, slicing, concatenation, and repetition to efficiently access and manage grouped data elements.

Enrollment/Roll No: 92600565007

Practical No. 13

Aim/ Objective:Python Programs to demonstrate the use of dictionary and various functions of it

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 3: Demonstrate use of Dictionary and various functions")
print("-" * 50)

# 1. Creating a Dictionary
student_dict = {
    'Name': 'Zara',
    'Age': 7,
    'Class': 'First',
    'Department': 'Cyber Security'
}
print("1. Initial Dictionary:")
print("  ", student_dict)

# 2. Accessing Elements
print("\n2. Accessing Elements:")
print("   Direct access student_dict['Name']:", student_dict['Name'])
print("   Using get() method student_dict.get('Age'):", student_dict.get('Age'))
print("   Using get() with default value:", student_dict.get('Grade', 'Not Assigned'))

# 3. Adding and Updating Elements
print("\n3. Adding & Updating Elements:")
student_dict['Age'] = 8                    # Updating existing key
student_dict['School'] = "DPS School"      # Adding new key-value pair
student_dict.update({'City': 'Rajkot', 'Marks': 95})  # Using update()
print("   Updated Dictionary:", student_dict)

# 4. Dictionary Methods (keys, values, items)
print("\n4. Dictionary Built-in Methods:")
print("   Keys:", list(student_dict.keys()))
print("   Values:", list(student_dict.values()))
```

```
print("   Key-Value Pairs (Items):", list(student_dict.items()))
print("   Total pairs (len):", len(student_dict))

# 5. Removing Elements (pop, popitem, del, clear)
print("\n5. Removing Elements:")
popped_val = student_dict.pop('Marks')
print(f"   After pop('Marks'): (Removed value: {popped_val})")
print("   Current Dict:", student_dict)

popped_item = student_dict.popitem()
print(f"   After popitem(): (Removed item: {popped_item})")
print("   Current Dict:", student_dict)

del student_dict['School']
print("   After del student_dict['School']:", student_dict)

# 6. Properties of Dictionary Keys
print("\n6. Properties of Dictionary Keys:")
# Property A: Duplicate keys are not allowed (last assignment overwrites)
dup_key_dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Vasu'}
print("   Duplicate keys overwrite previous value:", dup_key_dict)

# Property B: Keys must be immutable (hashable). Lists cannot be keys.
try:
    invalid_dict = {['Course']: 'Cyber Security'}
except TypeError as e:
    print("   Trying to use list as a key raises TypeError:", e)
    print("   Tuple (immutable) CAN be a key:")
    valid_dict = {('Course', 'Year'): 'MSc 2026'}
    print("   Valid Dict with Tuple key:", valid_dict)

# 7. Clearing Dictionary
student_dict.clear()
print("\n7. After clear():", student_dict)
```

Output:

```
\Python\practical\03_dictionary_program.py" ${command:pickArgs}
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
1. Initial Dictionary:
   {'Name': 'Zara', 'Age': 7, 'Class': 'First', 'Department': 'Cyber Security'}

2. Accessing Elements:
   Direct access student_dict['Name']: Zara
   Using get() method student_dict.get('Age'): 7
   Using get() with default value: Not Assigned

3. Adding & Updating Elements:
   Updated Dictionary: {'Name': 'Zara', 'Age': 8, 'Class': 'First', 'Department': 'Cyber Security', 'School': 'DPS Scho
ol', 'City': 'Rajkot', 'Marks': 95}

4. Dictionary Built-in Methods:
   Keys: ['Name', 'Age', 'Class', 'Department', 'School', 'City', 'Marks']
   Values: ['Zara', 8, 'First', 'Cyber Security', 'DPS School', 'Rajkot', 95]
   Key-Value Pairs (Items): [('Name', 'Zara'), ('Age', 8), ('Class', 'First'), ('Department', 'Cyber Security'), ('Scho
ol', 'DPS School'), ('City', 'Rajkot'), ('Marks', 95)]
   Total pairs (len): 7

5. Removing Elements:
   After pop('Marks'): (Removed value: 95)
   Current Dict: {'Name': 'Zara', 'Age': 8, 'Class': 'First', 'Department': 'Cyber Security', 'School': 'DPS School', '
City': 'Rajkot'}
   After popitem(): (Removed item: ('City', 'Rajkot'))
   Current Dict: {'Name': 'Zara', 'Age': 8, 'Class': 'First', 'Department': 'Cyber Security', 'School': 'DPS School'}
   After del student_dict['School']: {'Name': 'Zara', 'Age': 8, 'Class': 'First', 'Department': 'Cyber Security'}

6. Properties of Dictionary Keys:
   Duplicate keys overwrite previous value: {'Name': 'Vasu', 'Age': 7}
   Trying to use list as a key raises TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
   Tuple (immutable) CAN be a key:
   Valid Dict with Tuple key: {('Course', 'Year'): 'MSc 2026'}

7. After clear(): {}
```

Conclusion: This practical demonstrates the functionality of Python dictionaries, highlighting their key-value pair structure and the various methods available for efficient data management, including accessing, adding, updating, and removing elements, as well as the unique properties of dictionary keys.

Enrollment/Roll No: 92600565007

Practical No. 14

Aim/ Objective:Python Programs to create a function (Make your own assumptions).

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 4: Python Programs to create a function")
print("-" * 50)

# Function 1: Check if a number is Even or Odd
def check_even_odd(number):
    """Function to check whether a given number is even or odd."""
    if number % 2 == 0:
        return f"{number} is Even"
    else:
        return f"{number} is Odd"


# Function 2: Calculate factorial of a number
def calculate_factorial(n):
    """Function to compute the factorial of a non-negative integer."""
    if n < 0:
        return "Factorial does not exist for negative numbers"
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact


# Function 3: Calculate Student Grade based on marks percentage
def calculate_grade(marks):
    """Function to determine grade based on marks percentage."""
    if marks >= 90:
        return "Grade A+"
    elif marks >= 75:
        return "Grade A"
    elif marks >= 60:
```

```
        return "Grade B"
    elif marks >= 50:
        return "Grade C"
    elif marks >= 40:
        return "Grade D"
    else:
        return "Fail"


# Demonstrating Function Calls:
print("1. Testing Even/Odd Function:")
print("  ", check_even_odd(16))
print("  ", check_even_odd(7))

print("\n2. Testing Factorial Function:")
print("   Factorial of 5:", calculate_factorial(5))
print("   Factorial of 0:", calculate_factorial(0))

print("\n3. Testing Grade Calculation Function:")
print("   Marks: 85 ->", calculate_grade(85))
print("   Marks: 68 ->", calculate_grade(68))
print("   Marks: 32 ->", calculate_grade(32))
```

Output :

```
PS D:\MSc Cyber security\Python\practical>  & 'C:\Use
ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\deb
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 4: Python Programs to create a function
------------------------------------------------
1. Testing Even/Odd Function:
   16 is Even
   7 is Odd

2. Testing Factorial Function:
   Factorial of 5: 120
   Factorial of 0: 1

3. Testing Grade Calculation Function:
   Marks: 85 -> Grade A
   Marks: 68 -> Grade B
   Marks: 32 -> Fail
PS D:\MSc Cyber security\Python\practical>
```

Conclusion: This practical demonstrates the process of defining and calling custom functions in Python, showing how modular code blocks can be created to encapsulate specific logic—such as conditional checks, mathematical calculations, and data processing—for reusability and improved code organization.

Enrollment/Roll No: 92600565007

Practical No. 15

Aim/ Objective: Python Programs to demonstrate use of various arguments which can be passed to functions

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 5: Demonstrate various types of function arguments")
print("-" * 50)

# 1. Positional Arguments
# The arguments are passed to the function in the correct positional order.
def student_info(name, age, course):
    print(f"   Name: {name}, Age: {age}, Course: {course}")

print("1. Positional Arguments:")
student_info("Vasu Parmar", 21, "MSc Cyber Security")


# 2. Keyword Arguments
# Parameters are identified by argument names during the function call.
print("\n2. Keyword Arguments (Order does not matter):")
student_info(course="MSc Cyber Security", age=22, name="Aman")


# 3. Default Arguments
# If no argument is provided, the default value is used.
def greet_user(name, greeting="Welcome to Python Programming"):
    print(f"   Hello {name}, {greeting}!")

print("\n3. Default Arguments:")
greet_user("Vasu")                      # Uses default greeting
greet_user("Riya", "Good Morning")          # Overrides default greeting


# 4. Variable-Length Positional Arguments (*args)
```

```
# Allows passing any number of positional arguments as a tuple.
def calculate_total(*numbers):
    total = sum(numbers)
    print(f"   Passed Numbers: {numbers} -> Total Sum: {total}")

print("\n4. Variable-Length Positional Arguments (*args):")
calculate_total(10, 20)
calculate_total(10, 20, 30, 40, 50)


# 5. Variable-Length Keyword Arguments (**kwargs)
# Allows passing any number of keyword arguments as a dictionary.
def display_profile(username, **details):
    print(f"   Username: {username}")
    for key, value in details.items():
        print(f"     - {key.capitalize()}: {value}")

print("\n5. Variable-Length Keyword Arguments (**kwargs):")
display_profile("vasu_07", Role="Admin", Department="Cyber Security",
Status="Active")
```

Output:

```
PS D:\MSc Cyber security\Python\practical>  d:; cd 'd:\MSc Cy
\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.de
y\Python\practical\05_function_arguments_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 5: Demonstrate various types of function arguments
------------------------------------------------
1. Positional Arguments:
   Name: Vasu Parmar, Age: 21, Course: MSc Cyber Security

2. Keyword Arguments (Order does not matter):
   Name: Aman, Age: 22, Course: MSc Cyber Security

3. Default Arguments:
   Hello Vasu, Welcome to Python Programming!
   Hello Riya, Good Morning!

4. Variable-Length Positional Arguments (*args):
   Passed Numbers: (10, 20) -> Total Sum: 30
   Passed Numbers: (10, 20, 30, 40, 50) -> Total Sum: 150

5. Variable-Length Keyword Arguments (**kwargs):
   Username: vasu_07
     - Role: Admin
     - Department: Cyber Security
     - Status: Active
```

Conclusion: This practical demonstrates the versatility of Python function arguments, showcasing how positional, keyword, default, and variable-length arguments (*args and **kwargs) provide flexible ways to pass data into functions, thereby increasing code adaptability and readability.

Enrollment/Roll No: 92600565007

Practical No. 16

Aim/ Objective: Python Program to demonstrate the use of class

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 6: Demonstrate the use of Class and Objects")
print("-" * 50)


class Student:
    """This class demonstrates the definition of a Class, constructor, attributes, and methods in Python."""

    # Class Attribute (shared by all instances)
    college = "Marwadi University"


    # Constructor (__init__ method to initialize instance attributes)
    def __init__(self, name, enrollment_no, department):
        self.name = name                    # Instance attribute
        self.enrollment_no = enrollment_no    # Instance attribute
        self.department = department          # Instance attribute


    # Instance Method
    def display_details(self):
        print(f"   Name          : {self.name}")
        print(f"   Enrollment No : {self.enrollment_no}")
        print(f"   Department    : {self.department}")
        print(f"   College       : {self.college}")


    # Another Method
    def welcome_message(self):
        print(f"   Welcome {self.name} to the {self.department} department!")
```

```
# Printing Class Docstring
print("Class Documentation (__doc__):")
print("  ", Student.__doc__)
print("-" * 50)


# Creating Objects (Instances) of the Student class
print("Creating Object 1:")
student1 = Student("Vasu Parmar", "92600565007", "MSc Cyber Security")
student1.display_details()
student1.welcome_message()


print("\nCreating Object 2:")
student2 = Student("Rahul Patel", "MU2026007", "Faculty of Computer Applications")
student2.display_details()
student2.welcome_message()
```

Output:

```
y\Python\practical\06_class_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 6: Demonstrate the use of Class and Objects
------------------------------------------------
Class Documentation (__doc__):
   This class demonstrates the definition of a Class, constructor, attributes, and methods in Python.
------------------------------------------------
Creating Object 1:
   Name          : Vasu Parmar
   Enrollment No : 92600565007
   Department    : MSc Cyber Security
   College       : Marwadi University
   Welcome Vasu Parmar to the MSc Cyber Security department!

Creating Object 2:
   Name          : Rahul Patel
   Enrollment No : MU2026007
   Department    : Faculty of Computer Applications
   College       : Marwadi University
   Welcome Rahul Patel to the Faculty of Computer Applications department!
```

Conclusion:

This practical provides a clear demonstration of object-oriented programming in Python, showing how classes act as blueprints for creating objects with shared attributes and methods, effectively bundling data and functionality together to model real-world entities.

Enrollment/Roll No: 92600565007

Practical No. 17

Aim/ Objective: Python Program to demonstrate the concept of inner class

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 17: Demonstrate the concept of Inner Class")
print("-" * 50)

# Outer Class
class University:
    def __init__(self, university_name, city):
        self.university_name = university_name
        self.city = city

    def display_university_info(self):
        print(f"University Name : {self.university_name}")
        print(f"City            : {self.city}")

    # Inner Class (Class defined inside another class)
    class Student:
        def __init__(self, name, enrollment_no, department):
            self.name = name
            self.enrollment_no = enrollment_no
            self.department = department

        def display_student_info(self):
            print(f"Student Name    : {self.name}")
            print(f"Enrollment No   : {self.enrollment_no}")
            print(f"Department      : {self.department}")
```

```
# Creating an object of the Outer Class
univ = University("Marwadi University", "Rajkot")
print("--- Outer Class Details ---")
univ.display_university_info()

print("\n--- Inner Class Details (Created via Outer Class Instance) ---")
# Creating an object of the Inner Class using the Outer Class instance
student1 = univ.Student("Vasu Parmar", "92600565007", "MSc Cyber Security")
student1.display_student_info()

print("\n--- Inner Class Details (Direct Outer.Inner Instantiation) ---")
# Direct instantiation of the Inner class
student2 = University.Student("Rahul Patel", "MU2026007", "Faculty of Computer
Applications")
student2.display_student_info()
```

Output:

```
rogram.py
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 17: Demonstrate the concept of Inner Class
------------------------------------------------
--- Outer Class Details ---
University Name : Marwadi University
City            : Rajkot

--- Inner Class Details (Created via Outer Class Instance) ---
Student Name    : Vasu Parmar
Enrollment No   : 92600565007
Department      : MSc Cyber Security

--- Inner Class Details (Direct Outer.Inner Instantiation) ---
Student Name    : Rahul Patel
Enrollment No   : MU2026007
Department      : Faculty of Computer Applications
```

Conclusion:

The program successfully demonstrates the concept of an **inner class in Python** and shows how classes can be organized within another class.

Enrollment/Roll No: 92600565007

Practical No. 18

Aim/ Objective:Python program to demonstrate the use of methods.

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 8: Demonstrate the use of Methods in a Class")
print("-" * 50)

class StudentResult:
    """Class demonstrating methods with parameters, return values, and method chaining."""

    def __init__(self, name, enrollment_no):
        self.name = name
        self.enrollment_no = enrollment_no
        self.marks = []

    # Mutator Method (Setter to input marks)
    def set_marks(self, m1, m2, m3):
        self.marks = [m1, m2, m3]

    # Method with Return Value (Calculate total marks)
    def calculate_total(self):
        return sum(self.marks)
```

```
    # Method with Return Value (Calculate percentage)
    def calculate_percentage(self):
        total = self.calculate_total()
        return total / len(self.marks) if self.marks else 0

    # Method with Return Value (Calculate grade based on percentage)
    def calculate_grade(self):
        percentage = self.calculate_percentage()
        if percentage >= 80:
            return "Distinction (A+)"
        elif percentage >= 60:
            return "First Class (A)"
        elif percentage >= 40:
            return "Pass Class (B)"
        else:
            return "Fail"

    # Display Method (Calls other internal methods)
    def display_report_card(self):
        print(f"\n--- Report Card: {self.name} ---")
        print(f"Enrollment No : {self.enrollment_no}")
        print(f"Subject Marks : {self.marks}")
        print(f"Total Marks   : {self.calculate_total()} / {len(self.marks) * 100}")
        print(f"Percentage    : {self.calculate_percentage():.2f}%")
        print(f"Grade Awarded : {self.calculate_grade()}")


# Creating an object of StudentResult
student1 = StudentResult("Vasu Parmar", "92600565007")
student1.set_marks(85, 90, 88)
student1.display_report_card()

# Creating a second student object
```

```
student2 = StudentResult("Naeem Sheikh", "MU2026002")
student2.set_marks(72, 65, 78)
student2.display_report_card()
```

Output:

```
ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugp
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 8: Demonstrate the use of Methods in a Class
------------------------------------------------

--- Report Card: Vasu Parmar ---
Enrollment No : 92600565007
Subject Marks : [85, 90, 88]
Total Marks   : 263 / 300
Percentage    : 87.67%
Grade Awarded : Distinction (A+)

--- Report Card: Naeem Sheikh ---
Enrollment No : MU2026002
Subject Marks : [72, 65, 78]
Total Marks   : 215 / 300
Percentage    : 71.67%
Grade Awarded : First Class (A)
```

Conclusion: This practical illustrates how methods are used within classes to encapsulate operations on object data, demonstrating setters for state modification, methods with return values for calculations, and method-calling-method patterns to construct complex output reports.

Enrollment/Roll No: 92600565007

Practical No. 19

Aim/ Objective: Python program to demonstrate various types of methods.

Source Code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 9: Demonstrate various types of Methods")
print("-" * 50)

class Student:
    # Class Variable / Attribute
    university_name = "Marwadi University"
    total_students = 0

    def __init__(self, name, enrollment_no, marks):
        # Instance Variables
        self.name = name
        self.enrollment_no = enrollment_no
        self.marks = marks
        Student.total_students += 1

    # 1. Instance Method
    # Takes 'self' as the first parameter; can access and modify instance attributes
    def display_student_details(self):
        print(f"   Student Name  : {self.name}")
```

```
        print(f"   Enrollment No : {self.enrollment_no}")
        print(f"   Marks         : {self.marks}")
        print(f"   University    : {self.university_name}")

    # 2. Class Method
    # Takes 'cls' as the first parameter; decorated with @classmethod; can access/modify class variables
    @classmethod
    def get_university_info(cls):
        print(f"   [Class Method] University Name: {cls.university_name}")
        print(f"   [Class Method] Total Students Registered: {cls.total_students}")

    @classmethod
    def change_university_name(cls, new_name):
        cls.university_name = new_name
        print(f"   [Class Method] University name updated to: {cls.university_name}")

    # 3. Static Method
    # Does not take 'self' or 'cls'; decorated with @staticmethod; acts as a utility function
    @staticmethod
    def evaluate_result(marks):
        if marks >= 40:
            return "Pass"
        else:
            return "Fail"

    @staticmethod
    def is_valid_enrollment(enrollment_no):
        return len(str(enrollment_no)) > 0


# Creating Objects
student1 = Student("Vasu Parmar", "92600565007", 85)
```

```
student2 = Student("Rahul Patel", "MU2026007", 35)

print("1. Calling Instance Methods:")
print("--- Student 1 ---")
student1.display_student_details()
print("--- Student 2 ---")
student2.display_student_details()

print("\n2. Calling Class Methods:")
Student.get_university_info()

print("\n3. Calling Static Methods (Utility functions):")
print(f"   Student 1 ({student1.name}) Result:
{Student.evaluate_result(student1.marks)}")
print(f"   Student 2 ({student2.name}) Result:
{Student.evaluate_result(student2.marks)}")
print(f"   Is Enrollment Valid? {Student.is_valid_enrollment(student1.enrollment_no)}")
```

Output:

```
y\Python\practical\09_types_of_methods_program.py
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 9: Demonstrate various types of Methods
------------------------------------------------
1. Calling Instance Methods:
--- Student 1 ---
   Student Name  : Vasu Parmar
   Enrollment No : 92600565007
   Marks         : 85
   University    : Marwadi University
--- Student 2 ---
   Student Name  : Rahul Patel
   Enrollment No : MU2026007
   Marks         : 35
   University    : Marwadi University

2. Calling Class Methods:
   [Class Method] University Name: Marwadi University
   [Class Method] Total Students Registered: 2

3. Calling Static Methods (Utility functions):
   Student 1 (Vasu Parmar) Result: Pass
   Student 2 (Rahul Patel) Result: Fail
   Is Enrollment Valid? True
```

Conclusion: This practical demonstrates the use of methods within a class, showcasing how methods can be designed to perform specific operations such as modifying object attributes (mutators), performing calculations, and returning values to encapsulate functionality within a class structure.
Enrollment/Roll No: 92600565007

Practical No. 20

Aim/ Objective: Write a Python program to show method overloading by adding two numbers and three numbers using the same method name.

Source code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 10: Demonstrate Method Overloading (Adding 2 and 3 numbers)")
print("-" * 50)
```

```
class Addition:
    """Class to demonstrate method overloading using default arguments."""

    # Method Overloading using default argument (c=None)
    def add(self, a, b, c=None):
        if c is not None:
            result = a + b + c
            print(f"   Adding 3 numbers ({a} + {b} + {c}) = {result}")
            return result
        else:
            result = a + b
            print(f"   Adding 2 numbers ({a} + {b}) = {result}")
            return result


class AdvancedAddition:
    """Class to demonstrate method overloading using variable-length arguments
(*args)."""

    def add(self, *args):
        if len(args) == 2:
            result = args[0] + args[1]
            print(f"   Sum of 2 numbers {args}: {result}")
            return result
        elif len(args) == 3:
            result = args[0] + args[1] + args[2]
            print(f"   Sum of 3 numbers {args}: {result}")
            return result
        else:
            result = sum(args)
            print(f"   Sum of {len(args)} numbers {args}: {result}")
            return result
```

```
# Demonstration using Method 1 (Default Arguments):
print("Method 1: Overloading using Default Arguments:")
calc1 = Addition()
calc1.add(10, 20)         # Calling with 2 arguments
calc1.add(10, 20, 30)     # Calling with 3 arguments

# Demonstration using Method 2 (Variable-length Arguments):
print("\nMethod 2: Overloading using *args:")
calc2 = AdvancedAddition()
calc2.add(25, 75)         # Calling with 2 arguments
calc2.add(15, 30, 45)     # Calling with 3 arguments
calc2.add(5, 10, 15, 20)  # Calling with 4 arguments
```

Output:

```
ading_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 10: Demonstrate Method Overloading (Adding 2 and 3 numbers)
------------------------------------------------
Method 1: Overloading using Default Arguments:
   Adding 2 numbers (10 + 20) = 30
   Adding 3 numbers (10 + 20 + 30) = 60

Method 2: Overloading using *args:
   Sum of 2 numbers (25, 75): 100
   Sum of 3 numbers (15, 30, 45): 90
   Sum of 4 numbers (5, 10, 15, 20): 50
```

Conclusion: This practical demonstrates the use of methods within a class, showcasing how methods can be defined to encapsulate specific behaviors—such as setting values, performing calculations, and retrieving processed data—to build functional and structured objects for managing student report cards.

Enrollment/Roll No: 92600565007

Practical No. 21

Aim/ Objective: Write a Python program to show method overriding using a parent class Animal and a child class Dog.

Source code:

```
print("Name: Vasu Parmar")
print("Enrollment: 92600565007")
print("-" * 50)
print("Practical 11: Demonstrate Method Overriding using Animal and Dog")
print("-" * 50)
```

```
# Parent Class (Super Class / Base Class)
class Animal:
    def __init__(self, species="Unknown Animal"):
        self.species = species

    # Method to be overridden in child class
    def sound(self):
        print("   Animal sound : Generic animal sound")

    def display_info(self):
        print(f"   Species      : {self.species}")


# Child Class (Sub Class / Derived Class) inheriting from Animal
class Dog(Animal):
    def __init__(self, breed="Labrador"):
        # Calling parent constructor using super()
        super().__init__(species="Canine (Dog)")
        self.breed = breed

    # Overriding the sound() method of parent class Animal
    def sound(self):
        print(f"   Dog sound    : Bark! Woof Woof! (Breed: {self.breed})")

    # Additional child-specific method
    def fetch(self):
        print("   Dog action   : Fetching the ball...")


# 1. Creating an object of Parent Class (Animal)
print("1. Parent Class (Animal) Object:")
generic_animal = Animal()
generic_animal.display_info()
```

```
generic_animal.sound()

# 2. Creating an object of Child Class (Dog)
print("\n2. Child Class (Dog) Object (Method Overridden):")
my_dog = Dog(breed="German Shepherd")
my_dog.display_info()  # Inherited method from Animal
my_dog.sound()         # Overridden method in Dog
my_dog.fetch()         # Child's own method

# 3. Demonstrating Polymorphism via iteration
print("\n3. Polymorphic behavior:")
animals = [Animal(), Dog(breed="Golden Retriever")]
for idx, a in enumerate(animals, 1):
    print(f"   Object {idx}:")
    a.sound()
```

Output:

```
 Cyber security\Python\Unit 2 Practical\11_method_overriding_program.py'
Name: Vasu Parmar
Enrollment: 92600565007
------------------------------------------------
Practical 11: Demonstrate Method Overriding using Animal and Dog
------------------------------------------------
1. Parent Class (Animal) Object:
   Species      : Unknown Animal
   Animal sound : Generic animal sound

2. Child Class (Dog) Object (Method Overridden):
   Species      : Canine (Dog)
   Dog sound    : Bark! Woof Woof! (Breed: German Shepherd)
   Dog action   : Fetching the ball...

3. Polymorphic behavior:
   Object 1:
   Animal sound : Generic animal sound
   Object 2:
   Dog sound    : Bark! Woof Woof! (Breed: Golden Retriever)
```

Conclusion: